



Tipos de Algoritmos de Clasificación

XGBoost (Extreme Gradient Boosting)

Caso: Otorgamiento de crédito.

Juan Diego Ortiz Baquero

Gestor del conocimiento

Machine Learning [Conceptos]

Edison Gustavo Canon Varela

Universidad de Cundinamarca – Extensión Chía

Facultad de Ingeniería

Ingeniería de Sistemas y Computación

I. Resumen

Este informe presenta la implementación de un algoritmo de clasificación basado en XGBoost (Extreme Gradient Boosting) aplicado a un conjunto de datos de otorgamiento de crédito, en el que la variable objetivo es binaria (aprobación: Sí/No). El modelo se integra en una aplicación web desarrollada con Flask, utilizando plantillas Jinja2 y estilos Bootstrap para garantizar una visualización clara y estructurada. La solución incorpora una pestaña “Tipos de Algoritmos de Clasificación” en el menú principal, con dos submenús: Conceptos básicos y Caso práctico. En el primero, se expone una síntesis teórica de los principales algoritmos de clasificación mediante un mapa conceptual en MindMeister y referencias en formato APA 7; en el segundo, se carga el dataset de crédito, se entrena el modelo XGBoost, se evalúa mediante métricas (exactitud, reporte de clasificación y matriz de confusión) y se ofrece un formulario para predicciones “Sí/No” en tiempo real con umbral ajustable. El desarrollo se gestionó colaborativamente en GitHub mediante la rama *A8_TiposClasificacion*, con commits atómicos y descriptivos, Pull Request y merge a la rama principal.

Palabras clave: XGBoost, clasificación, Flask, aprendizaje supervisado, Jinja2, GitHub, APA.

II. Introducción

El aprendizaje supervisado permite entrenar modelos predictivos a partir de datos etiquetados, generando resultados con valor operativo en contextos reales. Dentro de esta categoría, los algoritmos de clasificación buscan asignar observaciones a clases discretas. En este

trabajo se implementa XGBoost, un algoritmo de ensamble basado en gradient boosting que combina múltiples árboles de decisión para mejorar la precisión y robustez del modelo.

- El caso práctico corresponde al otorgamiento de crédito, donde la variable objetivo es binaria (aprobación = Sí/No). El modelo se despliega en una aplicación web Flask que permite:
- Visualizar la descripción del dataset y las variables (numéricas y categóricas).
- Consultar métricas de evaluación: exactitud, reporte de clasificación y matriz de confusión.
- Ingresar nuevos valores mediante un formulario y obtener predicciones “Sí/No” con su probabilidad asociada y un umbral configurable.

III. Objetivos

3.1. Implementar un modelo de clasificación con XGBoost para predecir la aprobación de crédito a partir de variables numéricas y categóricas.

3.2. Integrar el modelo en una aplicación Flask con visualización estructurada y predicciones en tiempo real.

3.3. Incorporar una pestaña “Tipos de Algoritmos de Clasificación” con dos submenús: Conceptos básicos (mapa conceptual en MindMeister) y Caso práctico.

3.4. Gestionar el proyecto en GitHub mediante la rama A8_TiposClasificacion, con commits atómicos, Pull Request y merge a la rama principal.

3.5. Documentar el proceso con capturas, código y referencias en formato APA 7.

IV. Investigación – Conceptos Básicos de Regresión Logística

La clasificación es una técnica fundamental del aprendizaje supervisado cuyo objetivo es asignar una etiqueta o clase a cada observación a partir de un conjunto de variables independientes. Se utiliza cuando la variable objetivo es categórica, ya sea binaria (Sí/No), multiclase (varias categorías sin orden) u ordinal (categorías con un orden natural). Existen múltiples enfoques y algoritmos de clasificación, cada uno con ventajas, limitaciones y contextos de aplicación.

1. Modelos lineales

- Regresión logística: emplea la función sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

para transformar una combinación lineal de variables en una probabilidad entre 0 y 1.

- *Pros*: interpretable, rápido, adecuado para relaciones lineales.
- *Contras*: limitado frente a relaciones no lineales.
- SVM lineal: busca el hiperplano que maximiza el margen entre clases.
 - Controlado por el parámetro C, que regula la penalización de errores.
 - *Pros*: robusto en espacios de alta dimensión.
 - *Contras*: sensible a outliers y requiere escalado de variables.

2. Basados en distancia

- k-Nearest Neighbors (k-NN): clasifica una observación según la mayoría de sus k vecinos más cercanos.
 - Métricas comunes: distancia euclídea y Manhattan.
 - Requiere normalización de variables para evitar sesgos.
 - *Pros*: simple, sin entrenamiento explícito.
 - *Contras*: costoso en predicción y sensible al ruido.

3. Probabilísticos

- Naive Bayes: aplica el teorema de Bayes bajo el supuesto de independencia condicional entre variables:

$$P(C | x) \propto P(C) \prod P(x_i | C)$$

- Variantes: multinomial (texto), gaussiano (variables continuas).
- *Pros*: rápido, buen rendimiento en clasificación de texto.
- *Contras*: el supuesto de independencia rara vez se cumple en la práctica.

4. Árboles y Ensamblados

- Árbol de decisión: divide los datos en función de criterios como Gini o Entropía.
 - *Pros*: interpretable, maneja variables categóricas y numéricas.
 - *Contras*: propenso al sobreajuste si no se poda.
- Random Forest: combina múltiples árboles entrenados con *bagging* y muestreo aleatorio de variables.

- *Pros*: robusto, reduce varianza.
 - *Contras*: menos interpretable.
- Gradient Boosting: construye árboles secuenciales que corrigen los errores de los anteriores.
 - Controlado por tasa de aprendizaje y profundidad.
 - Variantes modernas: XGBoost, LightGBM, CatBoost.
 - *Pros*: alta precisión en problemas complejos.
 - *Contras*: requiere ajuste cuidadoso de hiperparámetros.

5. Redes Neuronales

- Perceptrón Multicapa (MLP): red de capas densas con funciones de activación como ReLU, Sigmoid o Tanh.
 - Salida softmax para clasificación multiclase.
 - *Pros*: gran capacidad de modelar relaciones no lineales.
 - *Contras*: requieren más datos, sensibles al escalado y al sobreajuste.

6. Tópicos transversales

- Preprocesamiento:
 - Escalado (Standard/MinMax) necesario para SVM, k-NN y MLP.
 - Codificación *one-hot* para variables categóricas.
 - Imputación de valores faltantes.

- Métricas de evaluación:
 - Exactitud (*accuracy*): proporción de aciertos.
 - Precisión (*precision*): proporción de positivos predichos que son correctos.
 - Recall (*sensibilidad*): proporción de positivos reales detectados.
 - F1-score: media armónica entre precisión y recall.
 - ROC-AUC: capacidad del modelo para discriminar entre clases.
 - Matriz de confusión: distribución de aciertos y errores por clase.



Imagen 1: Página web con los conceptos básicos de Tipos de Algoritmos de Clasificación, mapa conceptual y referencias en formato APA 7.

Referencias:

- Fernández, A. (2023). *Programar kNN desde cero en R*. Recuperado de <https://anderfernandez.com/blog/programar-knn-r/>
- IBM. (2024). *¿Qué es el algoritmo KNN?*. Recuperado de <https://www.ibm.com/mx-es/think/topics/knn>
- Wikipedia. (s.f.). *Clasificador bayesiano ingenuo*. En Wikipedia. Recuperado de https://es.wikipedia.org/wiki/Clasificador_bayesiano_ingenuo
- CienciaDeDatos.net. (2021). *Grid search Random Forest con OOB y early stopping*. Recuperado de <https://cienciadedatos.net/documentos/py36-grid-search-random-forest-out-of-bag-error-early-stopping>
- CienciaDeDatos.net. (s.f.). *Gradient Boosting con Python*. Recuperado de https://cienciadedatos.net/documentos/py09_gradient_boosting_python
- IBM. (s.f.). *¿Qué es XGBoost?*. Recuperado de <https://www.ibm.com/es-es/think/topics/xgboost>
- DataCamp. (2024). *Perceptrones multicapa: Guía completa*. Recuperado de <https://www.datacamp.com/es/tutorial/multilayer-perceptrons-in-machine-learning>
- LabEx. (s.f.). *Regularización en MLP*. Recuperado de <https://labex.io/es/tutorials/ml-multi-layer-perceptron-regularization-49214>
- Colab (Waissman, J.). (2024). *Preprocesamiento con ColumnTransformer y Pipeline*. Recuperado de <https://colab.research.google.com/github/ml-unison/ml-unison.github.io/blob/main/ejemplos/preprocesamiento.ipynb>

- DataCamp. (2024). *One-hot encoding en Python*. Recuperado de <https://www.datacamp.com/es/tutorial/one-hot-encoding-python-tutorial>
- Google Developers. (2025). *Accuracy, precision, recall y métricas relacionadas*. Recuperado de <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=es-419>
- TheMachineLearners. (s.f.). *Métricas de clasificación*. Recuperado de <https://www.themachinellearners.com/metricas-de-clasificacion/>
- OpenAI. (2025). ChatGPT (septiembre 22, versión GPT-5) [Modelo de lenguaje grande]. OpenAI. <https://chat.openai.com/>

V. Ejercicio Práctico – XGBoost Otorgamiento de crédito

Caso práctico de clasificación — XGBoost Otorgamiento de crédito				
Descripción del dataset • Variables numéricas: nivel_endeudamiento, ingresos_mensuales, edad (imputación por mediana). • Variables categóricas: historial_credito, tipo_empleo (imputación + one-hot). • Clase positiva: aprobacion = Si = 1; No = 0. • Split: 80/20 con estratificación y semilla fija. • Modelo: XGBoost con n_estimators=300, max_depth=4, learning_rate=0.08.				
Exactitud (Accuracy) 0.7				
Nota: La exactitud (accuracy) mide el porcentaje de predicciones correctas sobre el total de casos de prueba. Sin embargo, no siempre refleja bien el desempeño si las clases están desbalanceadas. Por eso también mostramos precisión, recall y F1-score en el reporte de clasificación.				
Reporte de clasificación				
Clase	Precisión	Recall	F1-score	Support
0	0.5	0.367	0.423	60.0
1	0.756	0.843	0.797	140.0

Imagen 2: Descripción del dataset, Exactitud y Reporte de clasificación

1.2. Matriz de Confusión

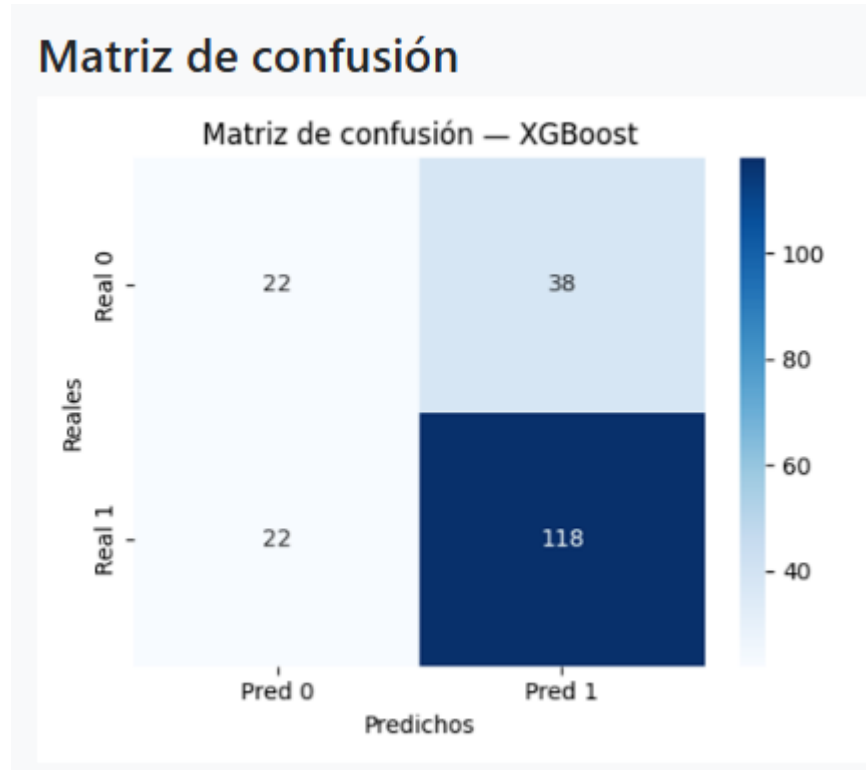


Imagen 3: Matriz de Confusión

1.5 Formulario de predicción

Predicción

Historial crediticio: Tipo de empleo:

Nivel de endeudamiento (0-1): Ingresos mensuales: Edad:

Threshold:

Predicción: Sí
Probabilidad: 0.9725
Con threshold=0.40, si reduces el umbral aumentas la sensibilidad (Recall) pero puedes bajar la precisión; si lo aumentas, ocurrirá lo contrario.

Imagen 4: Formulario de predicción con un ejemplo (Sí/No + probabilidad) desde la vista de la Página.

VI. Evidencias del Manejo del GitHub

1.1.Rama “A8_TiposClasificación”

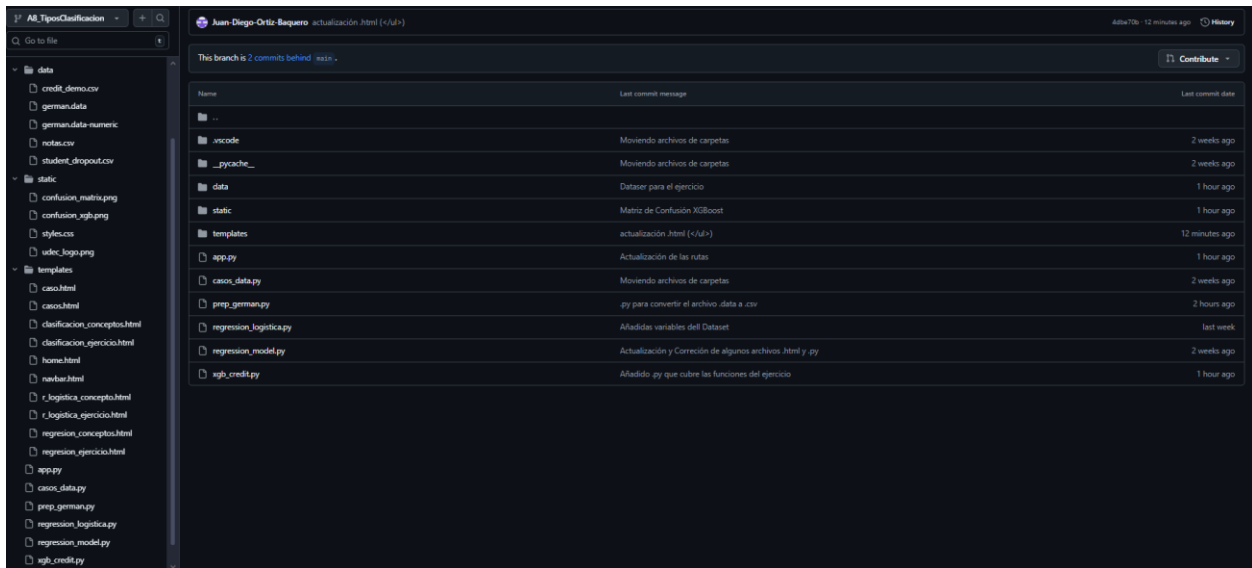


Imagen 7: Imagen donde se muestra el manejo de la rama “A8_TiposClasificación”

Pull Request y Merge

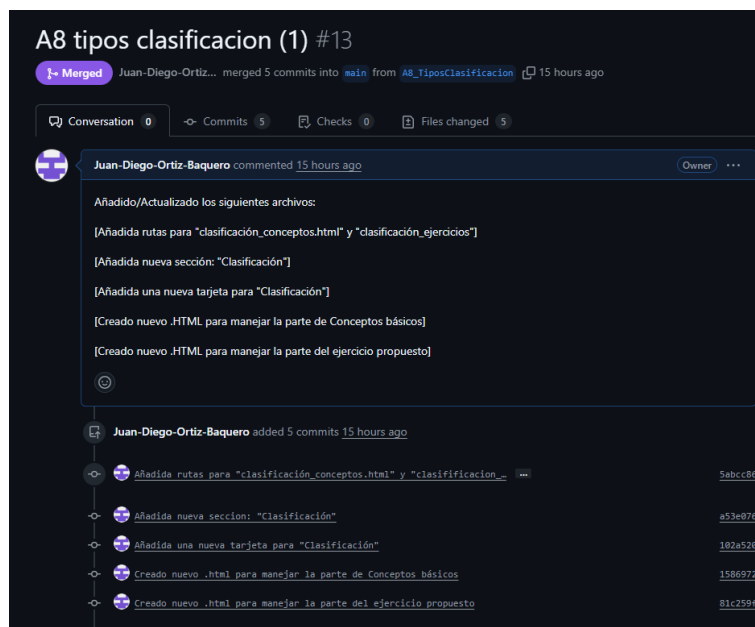


Imagen 8: Imagen donde se evidencia los Pull Request realizados para la actividad

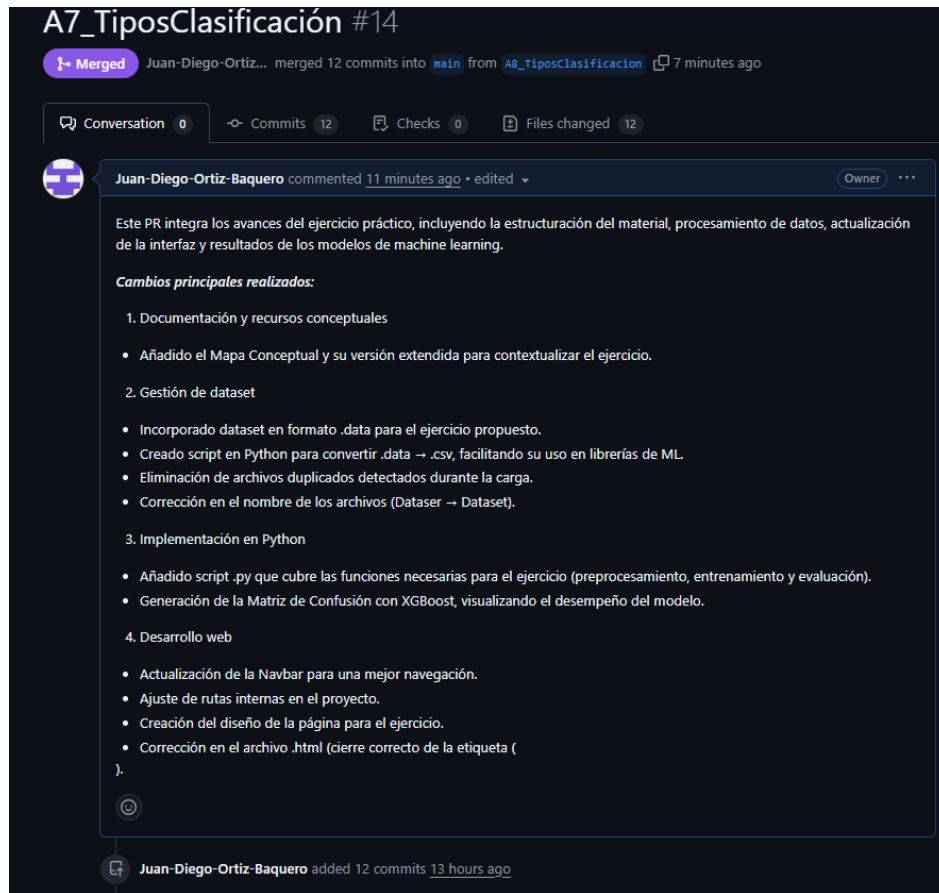


Imagen 9: Imagen donde se evidencia los Pull Request realizados para la actividad

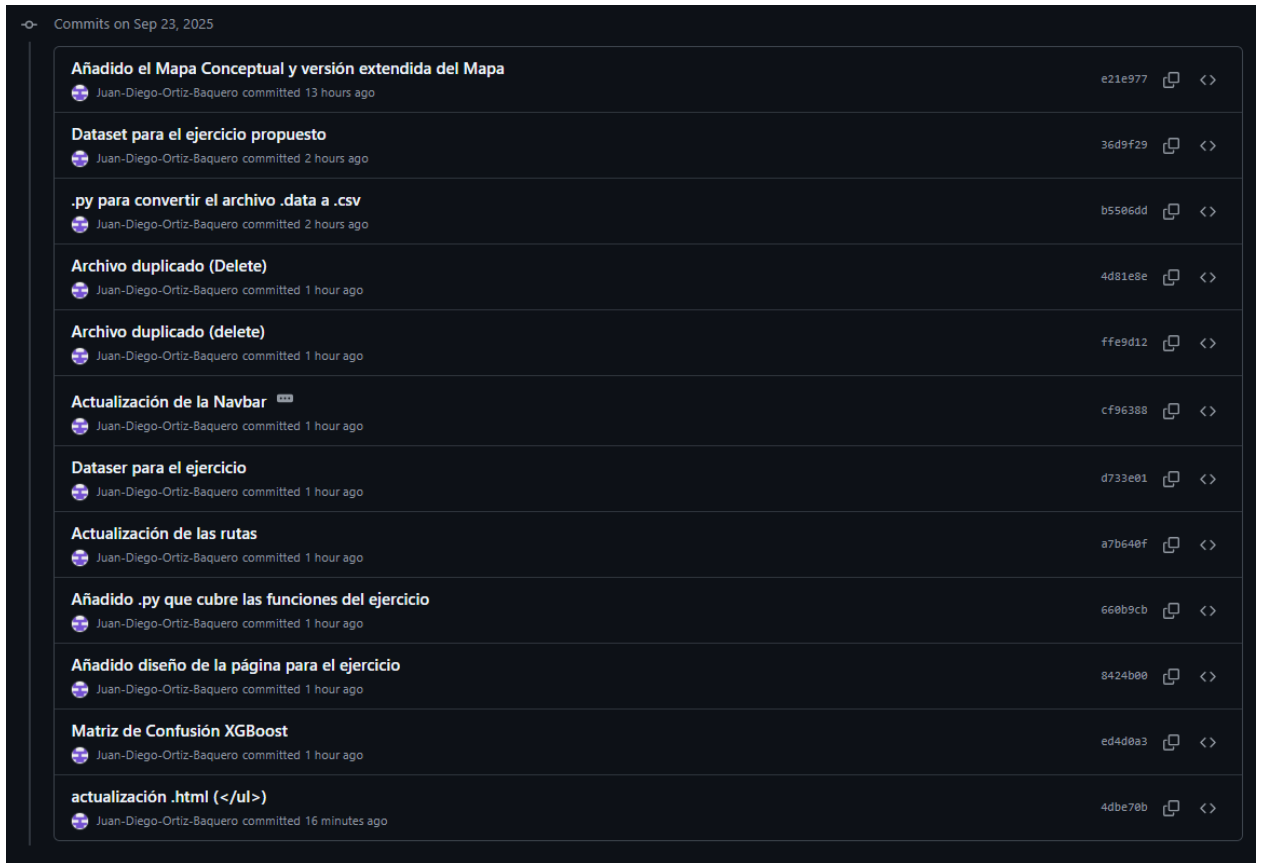


Imagen 10: Imagen donde se evidencia los Commit's

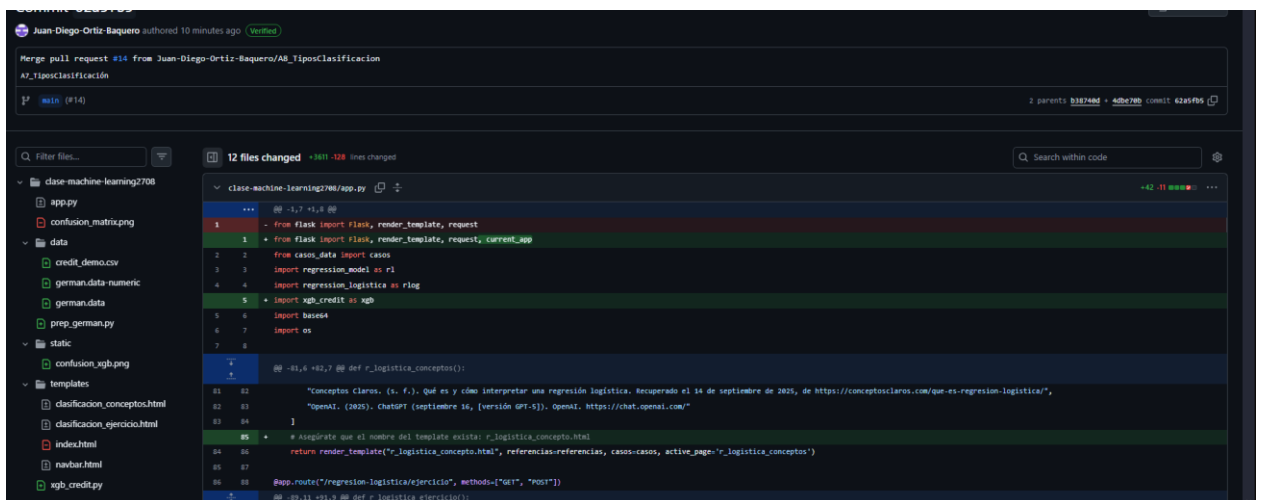


Imagen 11: Imagen donde se evidencia los el último merge

VII. Conclusiones

En este trabajo probé el algoritmo XGBoost para el caso de otorgamiento de crédito. El modelo funcionó bien porque puede manejar relaciones no lineales y variables diferentes sin mucho preprocesamiento. Al mover el umbral de decisión, noté que cambia el balance entre precisión y recall: con valores bajos se aprueban más casos pero aumentan los falsos positivos, y con valores altos pasa lo contrario. También vi que el desbalance de clases afecta las métricas, por lo que no basta con mirar la exactitud, sino que es mejor usar F1-score y la matriz de confusión. Como mejoras, se podrían ajustar parámetros como `scale_pos_weight`, recolectar más datos de la clase minoritaria, crear nuevas variables y calibrar las probabilidades. En general, aprendí que XGBoost es una buena opción para este tipo de problemas, siempre que se ajuste bien y se evalúe con varias métricas..

VIII. Referencias

- Hofmann, H. (1994). Statlog (German Credit Data) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5NC77>
- Fernández, A. (2023). Programar kNN desde cero en R. Recuperado de <https://anderfernandez.com/blog/programar-knn-r/>
- IBM. (2024). ¿Qué es el algoritmo KNN?. Recuperado de <https://www.ibm.com/mx-es/think/topics/knn>

- Wikipedia. (s.f.). Clasificador bayesiano ingenuo. En Wikipedia. Recuperado de https://es.wikipedia.org/wiki/Clasificador_bayesiano_ingenuo
- CienciaDeDatos.net. (2021). Grid search Random Forest con OOB y early stopping. Recuperado de <https://cienciadedatos.net/documentos/py36-grid-search-random-forest-out-of-bag-error-early-stopping>
- CienciaDeDatos.net. (s.f.). Gradient Boosting con Python. Recuperado de https://cienciadedatos.net/documentos/py09_gradient_boosting_python
- IBM. (s.f.). ¿Qué es XGBoost?. Recuperado de <https://www.ibm.com/es-es/think/topics/xgboost>
- DataCamp. (2024). Perceptrones multicapa: Guía completa. Recuperado de <https://www.datacamp.com/es/tutorial/multilayer-perceptrons-in-machine-learning>
- LabEx. (s.f.). Regularización en MLP. Recuperado de <https://labex.io/es/tutorials/ml-multi-layer-perceptron-regularization-49214>
- Colab (Waissman, J.). (2024). Preprocesamiento con ColumnTransformer y Pipeline. Recuperado de <https://colab.research.google.com/github/ml-unison/ml-unison.github.io/blob/main/ejemplos/preprocesamiento.ipynb>

- DataCamp. (2024). One-hot encoding en Python. Recuperado de <https://www.datacamp.com/es/tutorial/one-hot-encoding-python-tutorial>
- Google Developers. (2025). Accuracy, precision, recall y métricas relacionadas. Recuperado de <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=es-419>
- TheMachineLearners. (s.f.). Métricas de clasificación. Recuperado de <https://www.themachinellearners.com/metricas-de-clasificacion/>
- OpenAI. (2025). ChatGPT (septiembre 22, versión GPT-5) [Modelo de lenguaje grande]. OpenAI. <https://chat.openai.com/>

IX. Link Repositorio GitHub

- <https://github.com/Juan-Diego-Ortiz-Baquero/Machine-Learning>